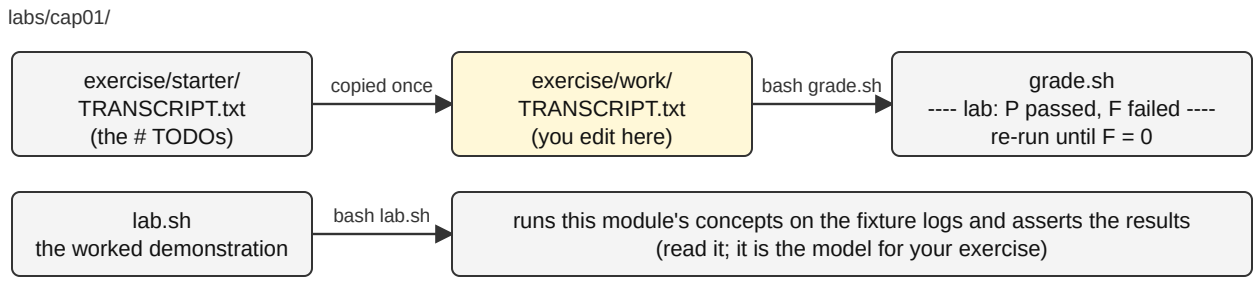
Capstone CAP-01 — Run fraud-clf Live with stackctl, and Hand It Over (with CAP-01R)

Topology



Environment Table

Everything happens in your own green tree from Lab 9, `labs/lab9-stackctl/exercise/work/`, on a host where `docker info` succeeds (Docker Desktop on macOS; Docker Engine on Ubuntu or inside WSL 2 Ubuntu on Windows 11). Two environments run side by side:

Environment	Project	Published port	Services
`test`	`fraudclf-test`	18080	`gateway` (nginx) → `serve` (fraud-clf FastAPI, built from `app/`)
`prod`	`fraudclf-prod`	18081	the same pair, a separate project, network and volumes

File	What it is	You produce it?
`bin/`, `lib/`, `compose/`, `tests/`, `tools/`	Your finished `stackctl` (LAB-09a/b)	Already done
`app/`	The inherited baseline — **untouched**	Never edited
`envs/test.conf`, `envs/prod.conf`	Environment files; after step 7 they carry the gateway digest	You append one value
`TRANSCRIPT.txt`	The `script` recording of steps 3–7	**Yes**
`MANIFEST.txt`	The unedited output of `tools/record-manifest.sh`	**Yes**
`RUNBOOK.md`	LAB-09c's handover, section 5 now filled from the manifest	**Yes**
`DECISIONS.md`	About 150 words, three answers	**Yes**

Objectives

- **Part 1:** Check the prerequisites and warm up (untimed).
- **Part 2:** Record the live lifecycle of `test`, then `prod`, then the destroy safety.
- **Part 3:** Tests, manifest and pins.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

- **Part 4:** Handover documents and the decisions note.
- **Part 5:** CAP-01R — one broken environment, under observation.

Background / Scenario

This is the summative check for LO-10, LO-11 and LO-12 and the point where Course 0 hands you to the rest of the program. In Lab 9 the mock driver proved your *control logic*; nothing from it counts as evidence here. Now you operate the program's project **for real**: a daemon, both environments, every command in the lifecycle, a prediction through the gateway, the two test suites, and the two handover documents — all recorded in a transcript. A second, shorter part puts one deliberately broken environment in front of you to locate, explain and fix while a trainer watches.

Passing CAP-01 is Course 2's Module 0 ("run the finished thing") done early, and your runbook's *Inherited findings* list is what that course's README tells you to keep.

Required Resources

- LAB-09a/b green (`---- lab9: 34 passed, 0 failed ----`), the LAB-09c runbook without `TODO`, A-10 passed.
- A host with a running Docker daemon and network access for the first image build and pull; `script` (or a terminal recorder).
- The module pages and the Docker documentation; AI assistance **for documentation lookup only** (say so in the note). Not `solution/`, not another learner's tree.

Validation status. The reference `stackctl` passes its mock suite 34/34 and the grader 34/34 in the authoring environment; the live procedure below was **designed** there, not run (no daemon) — verification task V-04. Your transcript is the first evidence either of us has that it works on *your* machine, which is exactly why it is required.

Part 1: Prerequisites and warm-up (untimed)

Step 1: Confirm the host.

```
cd labs/lab9-stackctl/exercise/work
docker info >/dev/null && echo "daemon ok"
bash ../grade.sh | tail -1
grep -c TODO RUNBOOK.md
```

Expected: `daemon ok`, `---- lab9: 34 passed, 0 failed ----`, `0`.

Step 2: Build once, outside the transcript.

```
bin/stackctl -e test up
CONFIRM_DESTROY=test bin/stackctl -e test destroy
```

The first `up` builds the serve image and pulls the gateway image — several minutes. If `up` times out while pip installs (exit 1), read `bin/stackctl -e test logs` and run `up` again. Nothing from this step goes in the transcript; it is what makes the timed run clean.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Part 2: The recorded lifecycle (timed, ≈35 min)

Step 1: Start the transcript.

Linux and WSL 2: `script -q TRANSCRIPT.txt`. macOS: `script TRANSCRIPT.txt`. Type `exit` at the very end to close it. If `script` is unavailable, copy your terminal into the file by hand — never edit what the commands printed.

Step 2: `test`, in this order.

```
bin/stackctl -e test validate
bin/stackctl -e test plan
bin/stackctl -e test up
bin/stackctl -e test check
bin/stackctl -e test status
bin/stackctl -e test plan            # must say: up would: nothing — converged
bin/stackctl -e test up              # must succeed and change nothing
bin/stackctl -e test logs | tail -5
curl -fsS http://127.0.0.1:18080/api/health; echo " status=$?"
curl -fsS -X POST http://127.0.0.1:18080/api/predict -H 'content-type: application/json'
-d '{"instances": [[120.0, 14, 0.2, 1]]}'; echo
```

Expected (designed; confirm on your machine): `validate → ok: test → project fraudclf-test on port 18080 (…)` plus the `warning:` about the tag-only gateway pin and the `note:` about the local serve image; `plan → up would: create project fraudclf-test (2 services)`; `up → up: done — 2 of 2 services running` (fast — the warm-up built the image); `check → check: http://127.0.0.1:18080/api/health → [{"status":"ok"}] ok`; `status → gateway` and `serve`; the second `plan → up would: nothing — converged`; the second `up → done — 2 of 2` with Compose reporting the containers as `Running`, not `Recreated`; `curl` health `→ {"status":"ok"} status=0`; predict `→ {"probabilities":[[p_legit,p_fraud]]}` — any two numbers summing to 1 are a correct answer from the baseline model (the exact values depend on training data regenerated at build and are not graded).

Step 3: `prod`, the same sequence with `-e prod` — while `test` is still up.

Then show the two are isolated:

```
docker compose ls
curl -fsS http://127.0.0.1:18081/api/health; echo " status=$?"
```

Expected: both `fraudclf-test` and `fraudclf-prod` listed as `running(2)`; `{"status":"ok"} status=0` on 18081.

Step 4: Destroy safety, in the transcript.

```
bin/stackctl -e test destroy; echo "status=$?"
CONFIRM_DESTROY=prod bin/stackctl -e test destroy; echo "status=$?"
CONFIRM_DESTROY=test bin/stackctl -e test destroy; echo "status=$?"
docker compose ls
CONFIRM_DESTROY=prod bin/stackctl -e prod destroy; echo "status=$?"
```

Expected: `destroy refused: set CONFIRM_DESTROY=test …` and `status=2`, twice; `destroy: project fraudclf-test removed (containers, network, volumes)` and `status=0`; `docker compose ls` still listing `fraudclf-prod` **running**; then `status=0` for `prod`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Symptom	Cause	Fix
`up` fails mentioning `address already in use`	Another app holds 18080/18081	Change `PORT` in the conf, `validate`, `plan`, `up`
`up` exits 1 after `WAIT_TIMEOUT`	Model still training at first build, or a slow pull	Read `logs`; retry; the warm-up should have prevented this
Second `plan` reports drift	A render-path difference you introduced (S09.4)	Explain it in D1; fix the render so both renders compare equal
`check` exits 1 right after `up`	Gateway up, path wrong, or port drift	`validate` shows `health GET <path>`; compare with what the app serves

Part 3: Tests, manifest and pins (≈15 min)

Step 1: Both suites, in the transcript.

```
bash tests/run.sh | tail -1
bash tests/live-smoke.sh test
```

Expected: `# 34 passed, 0 failed`; the smoke test builds (cached), brings up, checks, predicts once, proves convergence, destroys, and ends `== live smoke OK ==`.

Step 2: Manifest and pins.

```
bash tools/record-manifest.sh | tee MANIFEST.txt
```

Expected: per environment, one `IMAGE_GATEWAY … → nginx@sha256:…` line and one `IMAGE_SERVE fraudclf-serve:m0 → sha256:… (local build of APP_DIR/Dockerfile.course)` line. Append `@sha256:<digest>` **from that output** to `IMAGE_GATEWAY` in both conf files, then:

```
bin/stackctl -e test validate
bin/stackctl -e prod validate
exit            # closes the transcript
```

Expected: the tag-only `warning:` is gone; the `note:` about the locally built serve image stays (it has no digest to pin — it was never pushed; its ID exists only on the machine that built it).

Never type a digest or an image ID from a document, a chat answer or this manual. A copied value is a claim without an observation; the course treats it as fabricated evidence even when the string happens to be correct.

Part 4: Handover and the decisions note (≈10 min)

Step 1: `RUNBOOK.md`.

Complete from LAB-09c, with section 5 now filled from `MANIFEST.txt` and section 6 listing the inherited findings. `app/` stays untouched: the planted example keys in `app/config.py` are an inherited finding that belongs in section 6, not a fix you make here.

Step 2: `DECISIONS.md` — about 150 words in total, your own words.

D1. Your transcript shows `plan` saying `converged` and a second `up` that changed nothing. What in *your code* makes that true, and what in Compose? What change would `plan` *not* have seen?

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

D2. `destroy` refused twice before it worked. Name the two conditions it checks and say what each one protects against.

D3. Where did the gateway digest and the serve image ID in `MANIFEST.txt` come from, and why would copying either from the course page or a chat answer be a critical failure even if the string happened to be correct?

Rubric R-11 and hand-in

Deliver the `work/` tree (`bin/`, `lib/`, `envs/*.conf` with the recorded gateway digest, `app/` untouched, `tests/`, `RUNBOOK.md`, `MANIFEST.txt`), `TRANSCRIPT.txt` and `DECISIONS.md`. Levels **Not met** (0) · **Met** (1) · **Strong** (2), scored `weight × level / 2`. **Pass** = total ≥ 80 % of 26 **and** no *critical* criterion at Not met. CAP-01 may be re-attempted after remediation with a fresh transcript.

ID	Criterion	Evidence	Weight	Critical
R-11.1	Live operation of `test` then `prod`: `up` healthy, `check` ok, a prediction answered, `status`, both projects listed and isolated	transcript, Part 2	4	—
R-11.2	Health contract exact: `check` 0 live; mismatch and unreachable distinguished (3 vs 1)	transcript; mock suite	2	—
R-11.3	Validation precedes every mutation and `plan` never mutates	code + transcript + mock suite	3	**yes**
R-11.4	Idempotency: second `up` succeeds and `plan` reports converged	transcript	2	**yes**
R-11.5	Destroy safety: refuses without `CONFIRM_DESTROY=<this env>`; `down -v` scoped to the project; no global prune anywhere	transcript; code	3	**yes**
R-11.6	Tests: `tests/run.sh` 34/34 and `live-smoke.sh` OK, both in the transcript	transcript	3	—
R-11.7	Handover: `RUNBOOK.md` complete incl. inherited findings; `MANIFEST.txt` recorded on the operating machine; gateway pinned; `app/` untouched; no real secrets anywhere	files + transcript	3	**yes** (fabricated value, modified `app/`, or a real secret)
R-11.8	Decisions note: three answers that name the mechanism in your own words	`DECISIONS.md`	6	—

A flawless transcript with no decisions note scores 20/26 = 77 % — **not a pass**.

Part 5: CAP-01R — under observation (30 min)

Your trainer hands you a copy of your own tree with **one** environment altered: ports `28080`/`28081`, projects `fraudclf-r-test`/`fraudclf-r-prod`, and one injected fault somewhere in `envs/`, `compose/` or

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

the host state. With the trainer watching (live or on a recording):

1. Run the lifecycle until something fails.
2. **Say what you think the fault is before you fix it**, naming the exit code and the line of output that told you.
3. Fix it and verify with `check → [{"status":"ok"}] ok`.
4. Add the symptom and recovery to your `RUNBOOK.md` if it is not there yet.

Rubric R-12. *Critical:* the fault is **located and explained** from the tool's own output before the fix (R-12.1, weight 4); the fix is **verified by `check`** (R-12.2, weight 3). Non-critical: the runbook gains the case or already covered it (R-12.3, weight 2). Pass = ≥ 80 % and no critical at Not met. Fixing by trial and error without naming the cause fails R-12.1 even if the stack ends healthy; "fixing" `app/` to match a broken configuration is Not met.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.